

Student Placement Prediction System

Product Requirements Document (PRD)

1. Product Overview

A machine-learning-based analytics platform that predicts whether a student is likely to be Placed or Not Placed using information available before placement. The project dataset contains 100,000 student records and 26 columns.

2. Problem & Goal

Help students, colleges, and placement departments identify placement risk, understand important factors, and target preparation. Outputs: placement prediction, probability, risk category, contributing factors, and analytical recommendations.

3. Dataset & Target

Target: placement_status. Encode Placed=1 and Not Placed=0. Important features include CGPA, branch, college tier, internships, projects, certifications, coding, aptitude, communication, logical reasoning, hackathons, GitHub, LinkedIn, mock interview, attendance, backlogs, extracurriculars, leadership, volunteer experience, sleep hours, and study hours.

4. Data Leakage Rules

Remove student_id because it is an identifier. Remove salary_package_lpa because it is post-placement information and would create target leakage. Only pre-placement information may be used for prediction.

5. Data Processing

Inspect dimensions, types, missing values, duplicates, unique values, summary statistics, and target balance. Handle missing/inconsistent data appropriately. Encode categorical variables and scale numerical features where required. Use ColumnTransformer/Pipeline so training and inference use identical preprocessing.

6. Train/Test

Use an 80/20 split, random_state=42, with stratification. Keep test data unseen during training/model selection.

7. Required Models

Train and compare Logistic Regression, Random Forest, Support Vector Machine, and K-Nearest Neighbors. Do not assume Random Forest is the winner; select the final model from actual evaluation results.

8. Evaluation

Report accuracy, precision, recall, and F1-score for every model. Generate classification reports and confusion matrices. Pay special attention to recall for the Not Placed class.

9. Hyperparameter Tuning

Tune the strongest candidate using GridSearchCV or appropriate cross-validation. Record best parameters, cross-validation score, and before/after performance.

10. Explainability

Use model-supported feature importance or suitable explainability methods. Display actual important factors; never fabricate explanations. Correlation must not be presented as causation.

11. Prediction System

Create a student form using the actual model features. Validate inputs, apply the saved preprocessing pipeline, load the saved model, and return Placed/Not Placed plus model probability.

12. Risk Analysis

Provide configurable categories: High Placement Probability, Moderate Placement Probability, and Low Placement Probability. Clearly state that predictions are estimates, not guarantees.

13. Recommendations

Provide analytical suggestions around coding, projects, internships, aptitude, communication, interviews, academics, leadership, and other relevant areas based on the student's actual profile/model-supported factors.

14. Web Application

Build a modern, responsive professional analytics platform. Pages: Dashboard, Student Prediction, Prediction Result, Model Analytics, Feature Intelligence, and Data Explorer. Use actual dataset/model results; never hardcode ML statistics.

15. Backend

Prefer FastAPI with Pydantic/Uvicorn. Suggested endpoints: GET /api/health, GET /api/model-info, GET /api/metrics, GET /api/features, POST /api/predict, GET /api/analytics.

16. Architecture

Suggested structure: frontend/, backend/api/, backend/ml/, backend/preprocessing/, backend/models/, backend/services/, backend/utils/, data/, notebooks/, scripts/, tests/, README.md. Adapt when a better architecture is justified.

17. Performance

With 100,000 records, train offline and save model artifacts. Do not retrain on every prediction. Load the saved model for inference. Consider computational cost of SVM and KNN.

18. Testing & Security

Test data loading, leakage prevention, preprocessing, training, prediction, API validation, frontend validation, and error states. Do not expose secrets, internal paths, or raw stack traces.

19. Responsible AI

Never state that a student will definitely get placed. Explain that outputs are model estimates based on historical data. Avoid discriminatory recommendations or decisions based on protected characteristics.

20. No Fake Data

Do not invent model metrics, placement rates, feature importance, predictions, student records, or probabilities. All data-driven outputs must come from the actual dataset or trained model.

21. Development Workflow

Inspect PRD and CSV → validate schema/data → preprocess → train four models → evaluate → select candidate → tune → save model/preprocessor → build backend → build frontend → integrate → test → fix → document.

22. Technology

ML: Python, Pandas, NumPy, Scikit-learn, suitable visualization libraries. Backend: FastAPI, Pydantic, Uvicorn. Frontend: modern React/TypeScript with suitable styling and charts. Use REST APIs.

23. Definition of Done

Actual CSV is loaded; leakage is prevented; preprocessing works; all four required models train and are evaluated; best model is selected from actual results; tuning is completed; artifacts are saved; API and frontend work; prediction probability/risk/insights work; dashboards use real data; tests pass; README is complete.

Required Model Comparison

Model	Scaling	Role
Logistic Regression	Yes	Baseline / interpretable
Random Forest	Generally no	Tabular prediction / feature importance
SVM	Yes	Alternative classifier
KNN	Yes	Distance-based comparison